CSE 3211
Compiler Design

DR. K. M. AZHARUL HASAN

Compiler

- A **compiler** is a computer program (or set of programs) that
  - transforms source code written in a programming language (the *source language*) into another computer language (the *target language*, often having a binary form known as *object code*).
  - Is a translator program
- The **interpreter** takes one statement then translates and executes it and then takes another statement.
- While the compiler translates the entire program in one go and then executes it.

Compiler

- A compiler acts as a translator, transforming human-oriented programming languages into computer-oriented machine languages.
  - Ignore machine-dependent details for programmer

Compiler

- It is believed that the first compiler was written by Grace Hopper, in 1952, for the A-0 programming language.
- The FORTRAN team at IBM introduced the first complete compiler in 1957.
- Early compilers were written in assembly language. The first ***self-hosting*** compiler —
  - capable of compiling its own source code in a high-level language — was created in 1962 for Lisp at MIT.
- Building a self-hosting compiler is a ***bootstrapping problem***—
  - the first such compiler for a language must be compiled either by hand or by a compiler written in a different language, or compiled by running the compiler in an interpreter.

Why Do We Study Compilers?

- To design a translator
- Influences on programming language design
- Influences on computer design
- Compiling techniques are useful for software development
  - Parsing techniques are often used
  - Learn practical data structures and algorithms
  - Basis for many tools such as text formatters, structure editors, design verification tools,…

Writing a compiler requires an understanding of almost all important CS subfields

Learning outcome as a whole

- A great chance to "concretize" your knowledge of **Data Structures and Algorithms** and it teaches you to apply those concepts in non trivial scenarios.
- It helps you to develop skills which have so much application in other areas like
  - Text and language processing;
  - Specially translate one form to another
  - Getting an insight into how your own programs can be optimized;
  - Getting new ideas about better computer architectures.

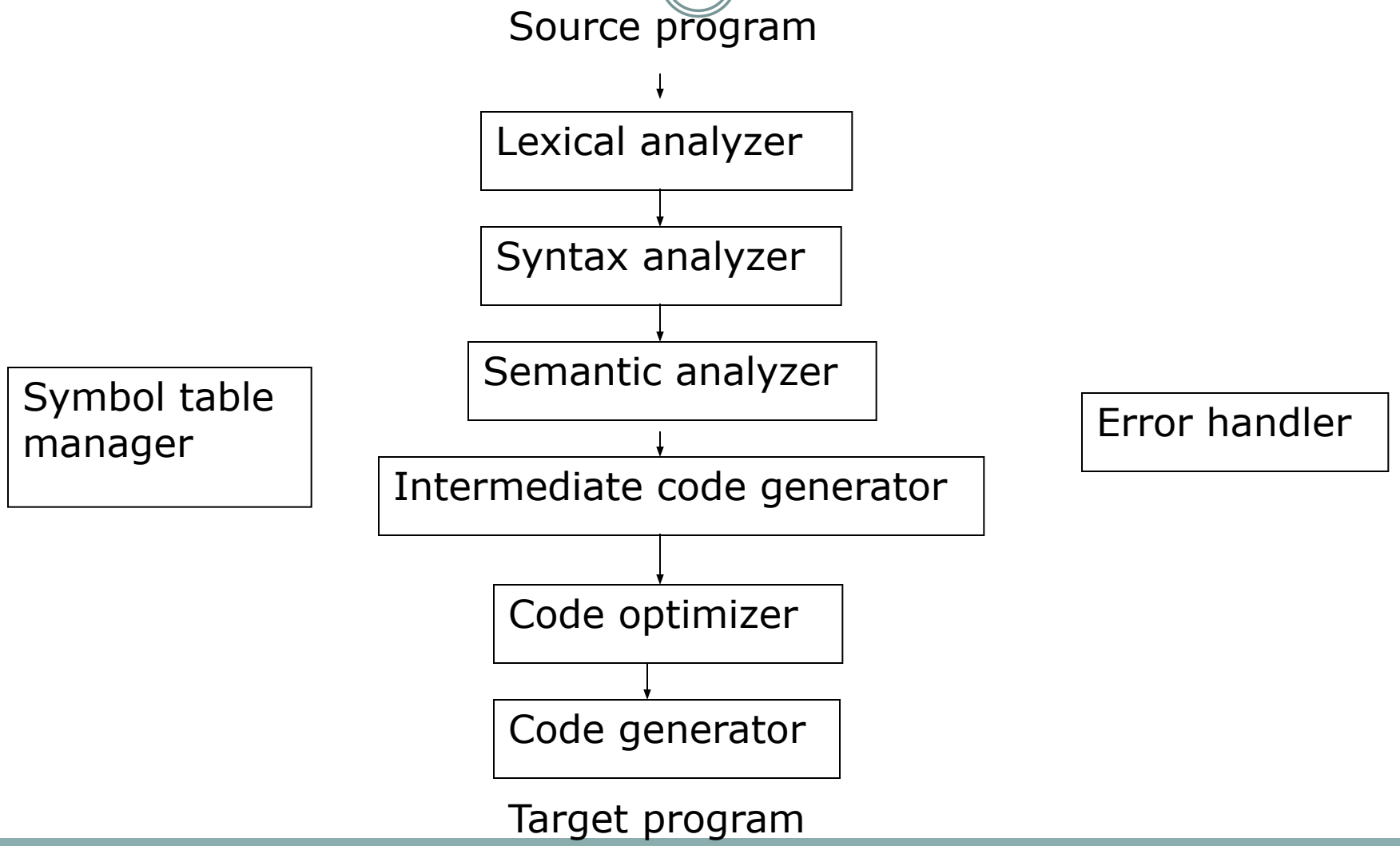
Even if you don't have to write a programming language ever in your life, a **Compiler design** course makes you a better programmer.

Specific Learning outcome

On successful completion of the course students will be able to

1. Organize the lexical, syntactic and semantic analysis into meaningful phases for a compiler to undertake language translation.
2. Design syntax and semantic analyser to develop a scanner and parser using scanner and parser generator tools.
3. Formulate parsing techniques for designing a compiler.
4. Apply the techniques for intermediate code and machine code optimization.
5. Explain the performance of a program in terms of speed and space using optimization techniques.
6. Design and implement translator to convert a language from one form to another.

Phases of a Compiler



Summary : Compiler Design

- **Lexical analysis –**
  - You will get good hands on experience with **Regular Expressions and will be important** when you deal with
    - big text files;
    - automating routine text processing tasks.
- **Syntax Analyzer-**
  - you're going to learn the most basic parsing algorithms
  - Parsing is important in
    - text and language processing tasks; if you're trying to interpret tweets or sentences, etc.

Summary : Compiler Design

- **Semantic analysis –**
  - At this stage, you'll actually start to understand the programming languages that you use even in your day to day life.
  - Understanding various constraints related to assignment, allocation; and you'll be able to appreciate why different programming languages do different things in different ways.

- **Intermediate code generation-**
  - Generates middle level code according to the semantics.
  - You will learn different platform independent codes

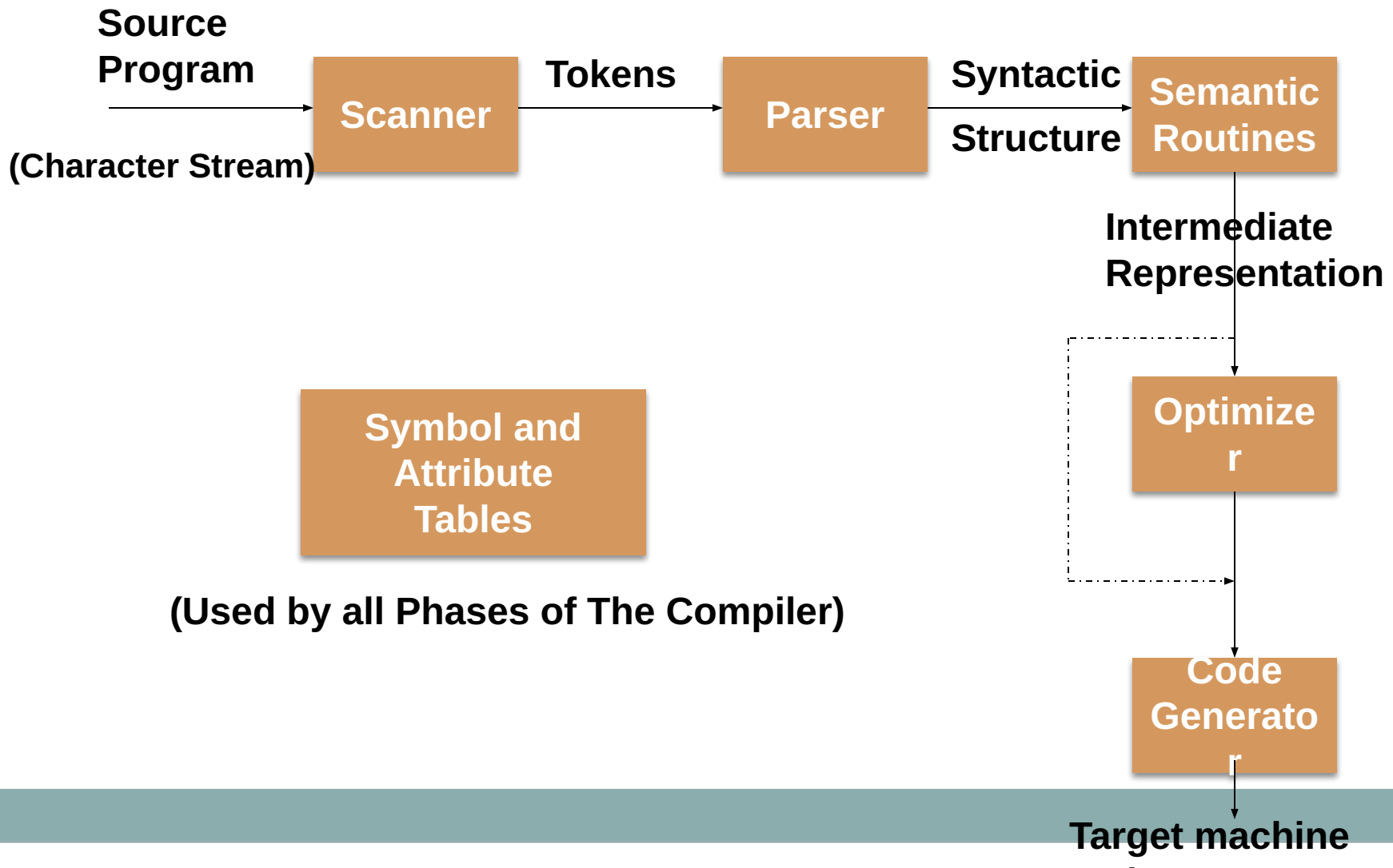
Code optimization –

- you will be introduced to various **Optimization** techniques in code generation.
- Once you are introduced this; you will also be more critical of the code you write for other works.
- You will be more conscious about used/unused variables and will develop the ability to think in terms of **program analysis and optimization** techniques which help you write more efficient code more quickly.

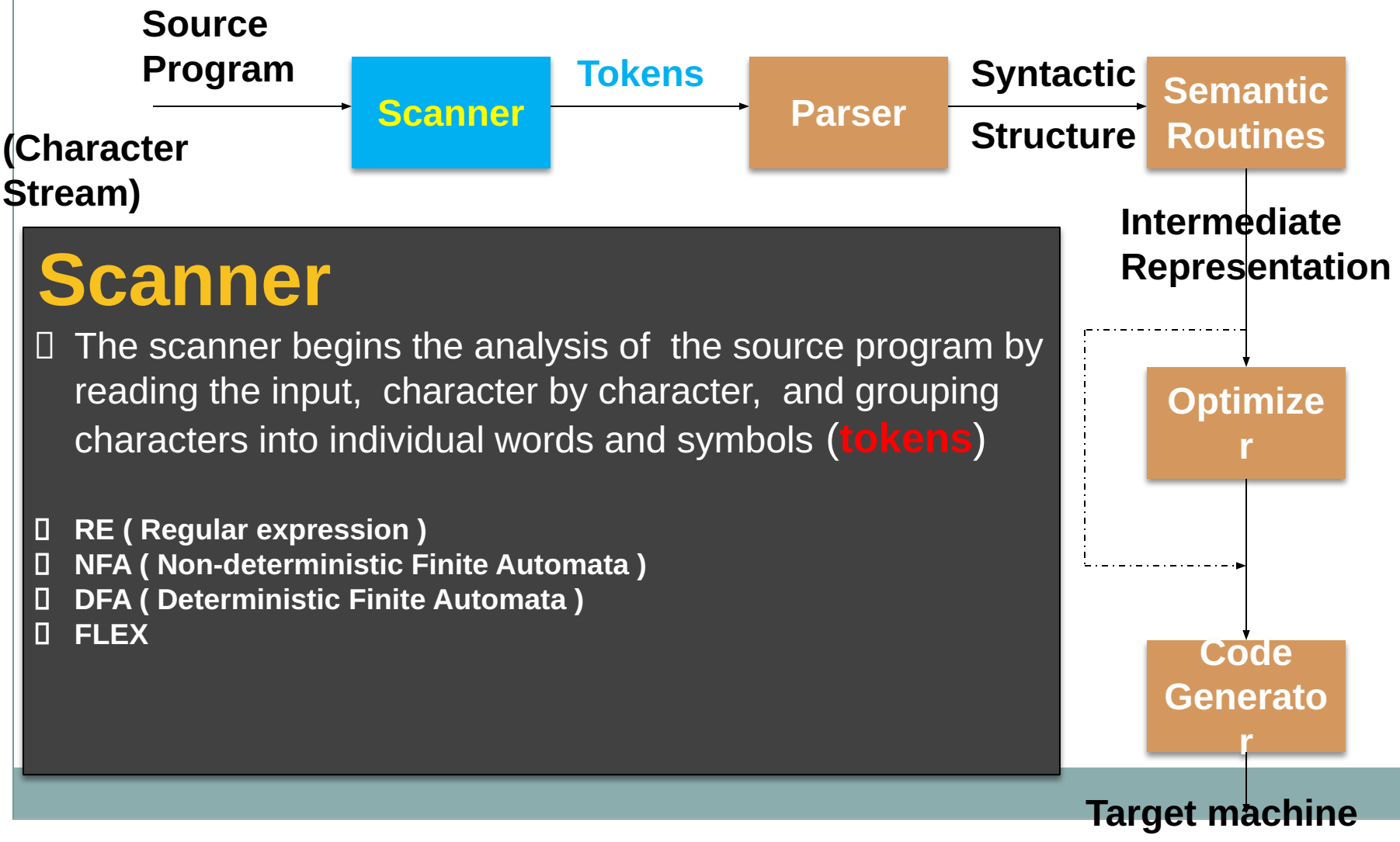
Summary : Compiler Design

- **Code generation –**
  - you learn to generate the **Machine Code**.
  - This forces you to learn Assembly Language;
    - how the computer interprets and executes instructions;
    - an overview of applying basic Computer Architecture concepts.

The Structure of a Compiler



The Structure of a Compiler

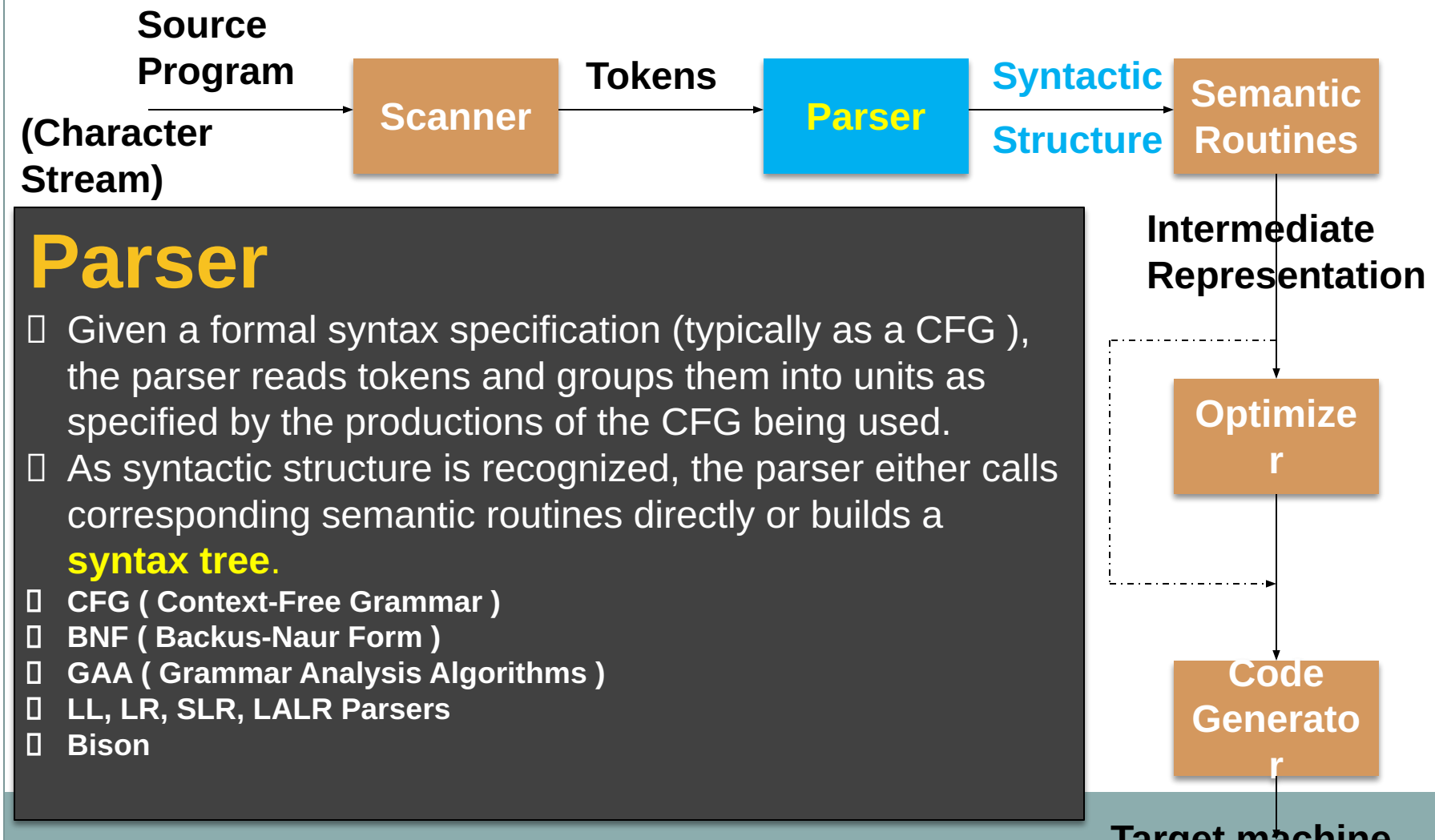


Scanner

- The scanner begins the analysis of the source program by reading the input, character by character, and grouping characters into individual words and symbols (tokens)

- RE (Regular expression)
- NFA (Non-deterministic Finite Automata)
- DFA (Deterministic Finite Automata)
- FLEX

The Structure of a Compiler

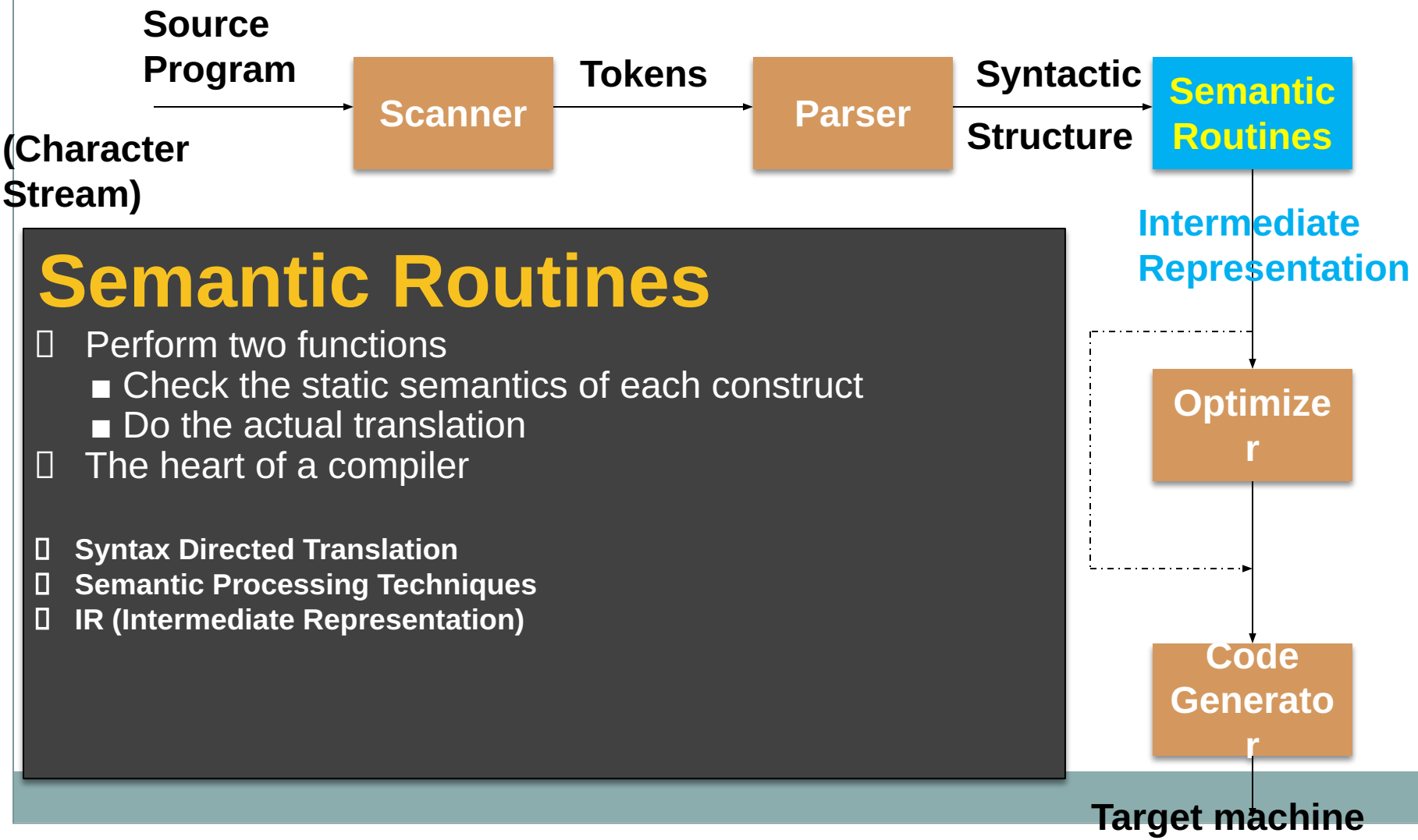


Parser

- Given a formal syntax specification (typically as a CFG), the parser reads tokens and groups them into units as specified by the productions of the CFG being used.
- As syntactic structure is recognized, the parser either calls corresponding semantic routines directly or builds a **syntax tree**.

- **CFG (Context-Free Grammar)**
- **BNF (Backus-Naur Form)**
- **GAA (Grammar Analysis Algorithms)**
- **LL, LR, SLR, LALR Parsers**
- **Bison**

The Structure of a Compiler

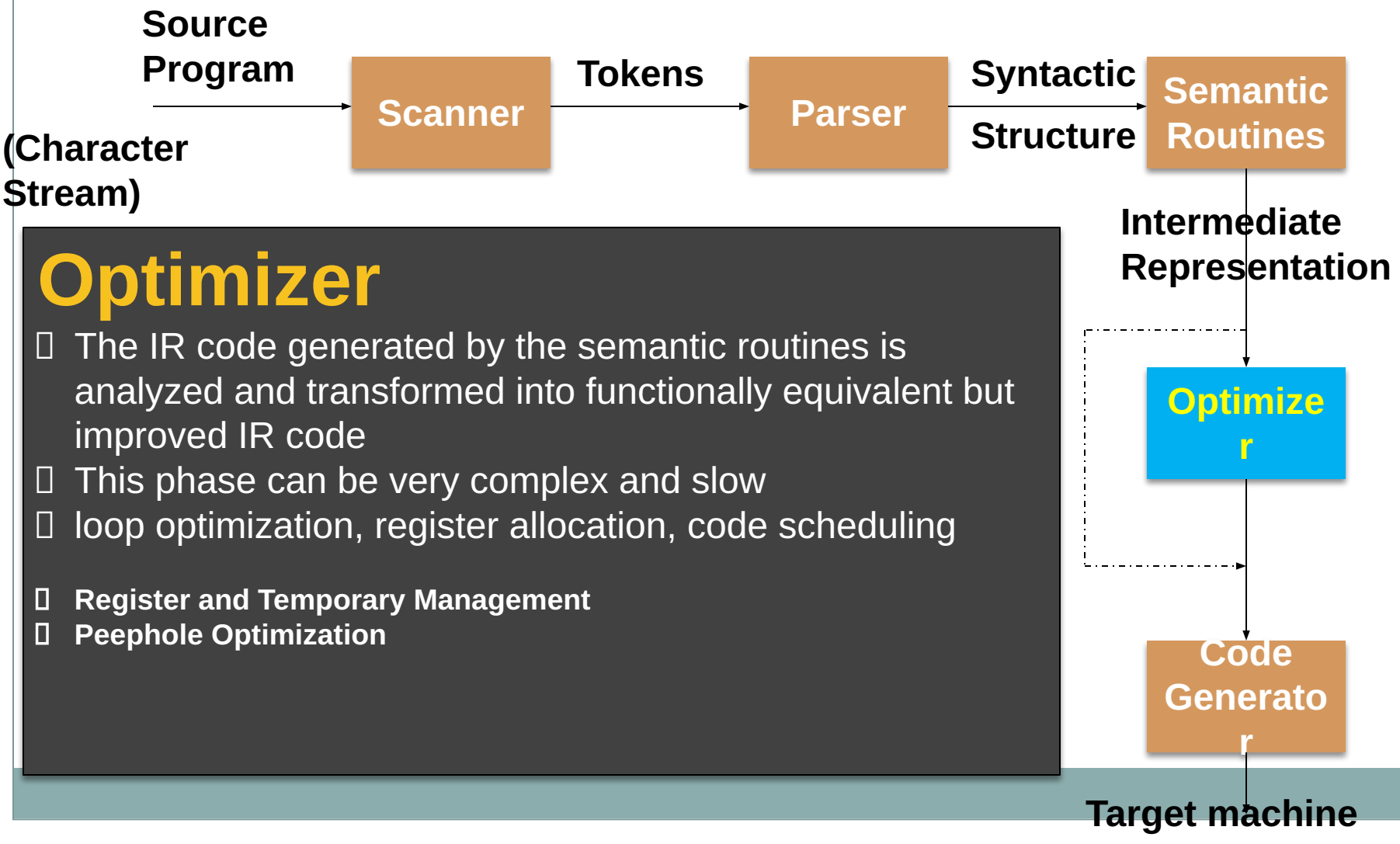


Semantic Routines

- Perform two functions
  - Check the static semantics of each construct
  - Do the actual translation
- The heart of a compiler

- **Syntax Directed Translation**
- **Semantic Processing Techniques**
- **IR (Intermediate Representation)**

The Structure of a Compiler

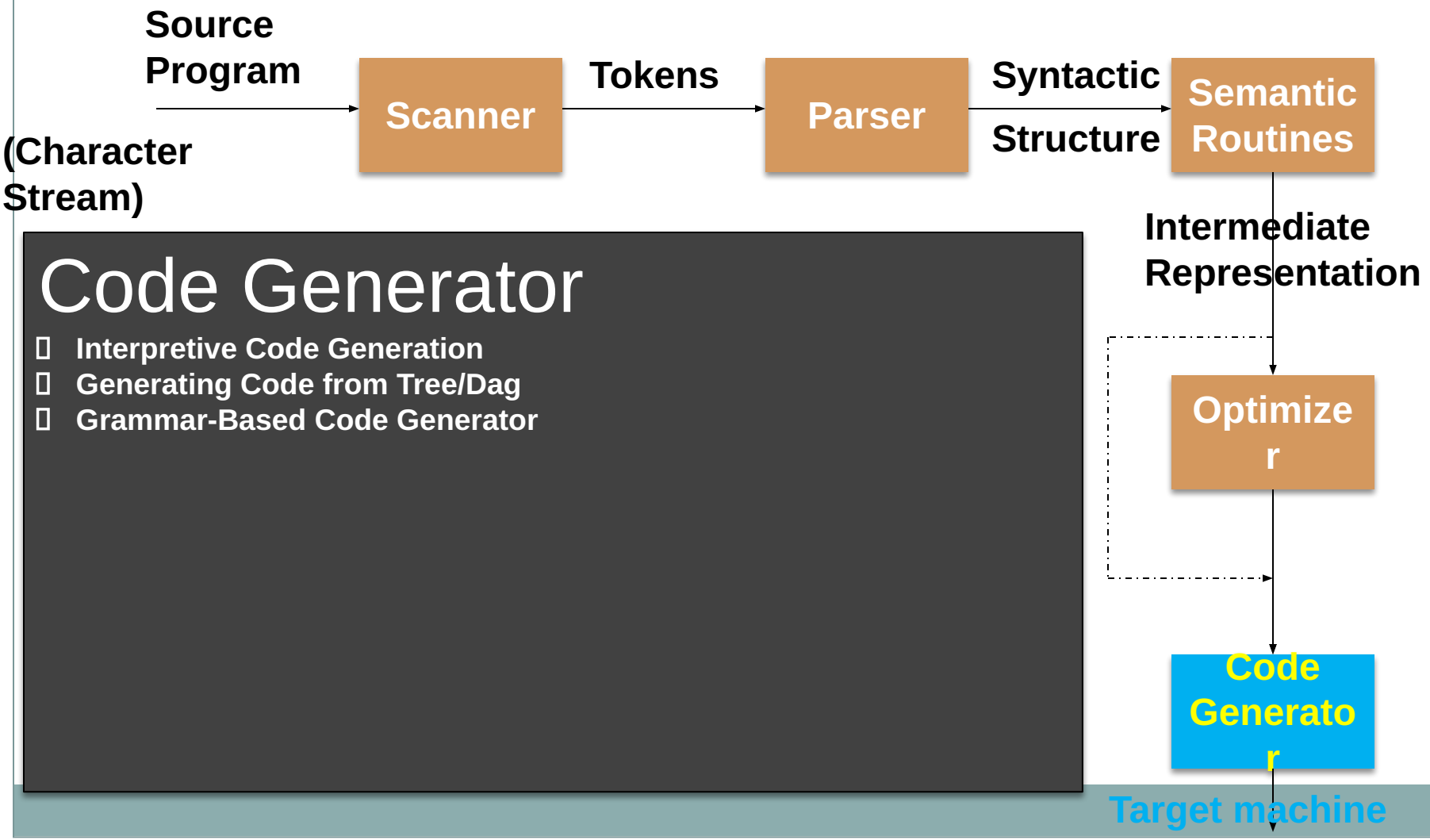


Optimizer

- The IR code generated by the semantic routines is analyzed and transformed into functionally equivalent but improved IR code
- This phase can be very complex and slow
- loop optimization, register allocation, code scheduling

- **Register and Temporary Management**
- **Peephole Optimization**

The Structure of a Compiler



Code Generator

- Interpretive Code Generation
- Generating Code from Tree/Dag
- Grammar-Based Code Generator

The Structure of a Compiler


```
position := initial + rate * 60
```

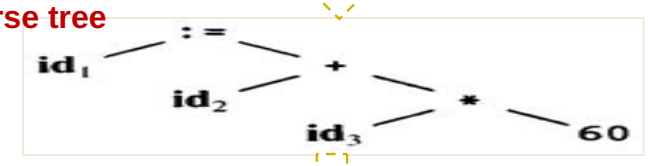
Scanner
[Lexical Analyzer]

Tokens

$id_1 := id_2 + id_3 * 60$

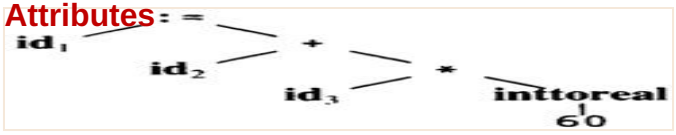
Parser
[Syntax Analyzer]

Parse tree



Semantic Process
[Semantic analyzer]

Abstract Syntax Tree w/ Attributes



Symbol Table

1	position	...
2	initial	...
3	rate	...
4		

Code Generator
[Intermediate Code Generator]

Non-optimized Intermediate Code

```
temp1 := inttoreal(60)
temp2 := id3 * temp1
temp3 := id2 + temp2
id1 := temp3
```

Code Optimizer

Optimized Intermediate Code

```
temp1 := id3 * 60.0
id1 := id2 + temp1
```

Code Generator

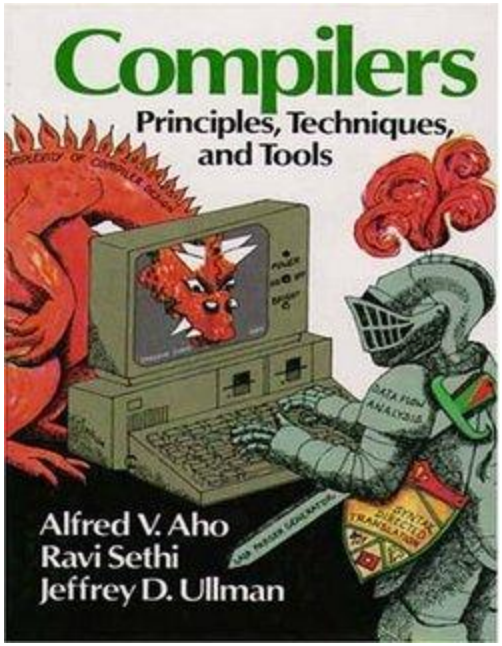
Target machine code

```
MOVF id3, R2
MULF #60.0, R2
MOVF id2, R1
ADDF R2, R1
MOVF R1, id1
```

References

- **1. Lecture Slides and Materials**

- **2. A. Aho, R. Sethi and D. Ullman, "*Compilers: Principles, Techniques and Tools*", *Addison Wesley*.**
  - **The Dragon faced book**
  - **Old book, but still sufficient**
  - **Copies can be found in the library**



- **3. A. I. Holub, "Compiler Design in C", Prentice Hill.**

Have nice time

with

Compiler Design!!!